Midterm Mastery 2

Ziang Chen

Contents

1. Instability of large file transfer
2. Loss on purpose
3. Scaling Behavior



Instability of Large File Transfer

Files tested:

Name	Type	Last modified	Size	Storage class
1b.txt	txt	November 2, 2026, 02:54:24 (UTC-08:00)	1.0 B	Standard
64b.txt	txt	November 2, 2026, 02:54:25 (UTC-08:00)	64.0 B	Standard
512b.txt	txt	November 2, 2026, 02:54:26 (UTC-08:00)	512.0 B	Standard
4k.txt	txt	November 2, 2026, 02:54:25 (UTC-08:00)	4.0 KB	Standard
32k.txt	txt	November 2, 2026, 02:54:25 (UTC-08:00)	32.0 KB	Standard
64k.txt	txt	November 2, 2026, 03:23:28 (UTC-08:00)	64.0 KB	Standard
128k.txt	txt	November 2, 2026, 03:22:56 (UTC-08:00)	128.0 KB	Standard
256k.txt	txt	November 2, 2026, 03:22:56 (UTC-08:00)	256.0 KB	Standard
2m.txt	txt	November 2, 2026, 02:54:25 (UTC-08:00)	2.0 MB	Standard
4m.txt	txt	November 2, 2026, 02:54:25 (UTC-08:00)	4.0 MB	Standard

Result:

	A	B	C	D
1	Fize Size	Requests	Fails	Avg Response Time
2	1 B	53	0	2,102
3	64 B	54	0	2,090
4	512 B	54	0	2,097
5	4 KB	54	0	2,084
6	32 KB	39	0	2,929
7	64 KB	32	2	3,636
8	128 KB	12	4	8,918
9	256 KB	9	5	10,836
10	2 MB	4	4	23,881
11	4 MB	2	2	34,932
12				

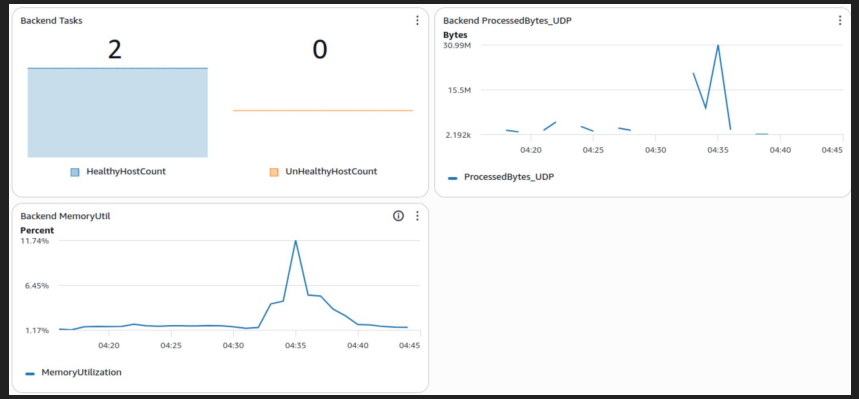
Failures:

```
November 2, 2026, 04:35    [GIN] 2026/11/02 - 11:35:40 | 504 | 34.76s | 154.5.118.59 | GET "/files?name=4m.txt"

                           Error #01: gave up with chunks still missing: [4267 4268 4269 4270 4271 4272 4273 4274
                           10569 10570 10571 10572 10573 10574 10575 10576 10577 10578 10579 10580 10581 10582 105
```

Instability of Large File Transfer

Why? Let’s see CloudWatch:

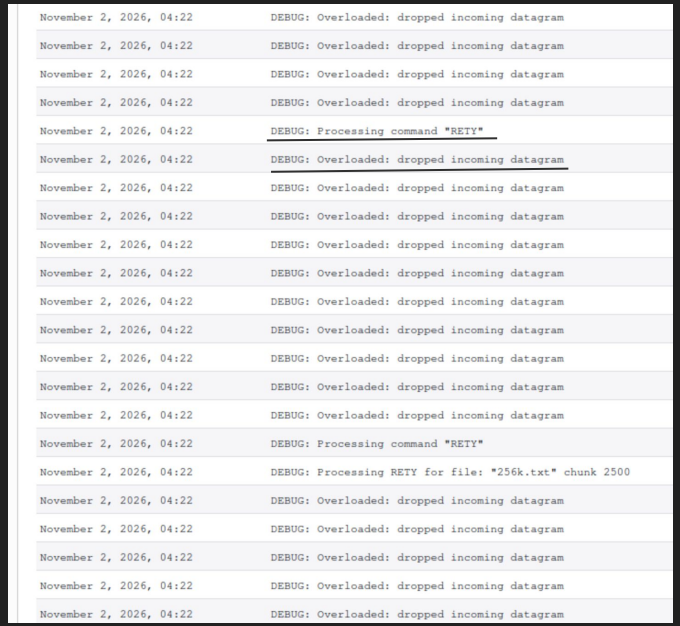


The service was doing fine, so it can only be the problem with the code itself.

We need to see the logs to diagnose.

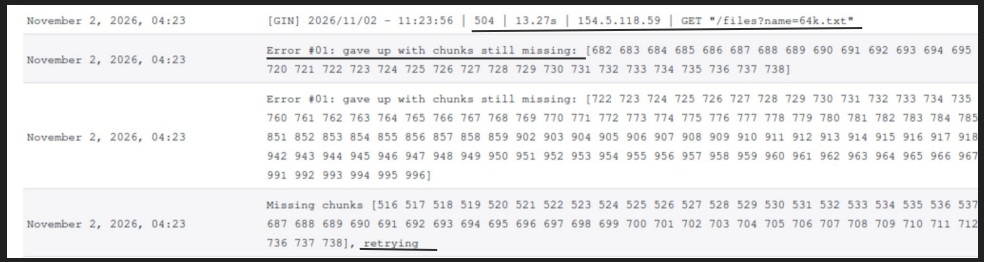
Instability of Large File Transfer

Backend Logs:



November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Processing command "RETY"
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Processing command "RETY"
November 2, 2026, 04:22	DEBUG: Processing RETY for file: "256k.txt" chunk 2500
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram
November 2, 2026, 04:22	DEBUG: Overloaded: dropped incoming datagram

Frontend Logs:



November 2, 2026, 04:23	[GIN] 2026/11/02 - 11:23:56 | 504 | 13.27s | 154.5.118.59 | GET "/files?name=64k.txt"
November 2, 2026, 04:23	Error #01: gave up with chunks still missing: [682 683 684 685 686 687 688 689 690 691 692 693 694 695 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738]
November 2, 2026, 04:23	Error #01: gave up with chunks still missing: [722 723 724 725 726 727 728 729 730 731 732 733 734 735 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 851 852 853 854 855 856 857 858 859 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 991 992 993 994 995 996]
November 2, 2026, 04:23	Missing chunks [516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 736 737 738], retrying

We can see 2 things:

1. Frontend returns 504 after giving up on getting missing chunks
2. A lot of datagrams were dropped because of overload

Instability of Large File Transfer

Reason:

1. Big files = more chunks & datagrams = spike in traffic = more missing chunks
2. Gateway sends “RETY” for each missing chunk
3. RETY requests read the same file from S3, which takes time
4. Queue for retries is only 32 long, all the extra retries will be dropped
5. Gateway retries for missing chunk 5 times.
6. Still no response after 5 retry rounds, return 504.

Loss on purpose

To further discover how loss acts on the service quality, I will introduce mandatory loss on the deploy.

This experiment runs on 4kb size documents, 5 workers, for 5 minutes.

At 1% send & receive loss:

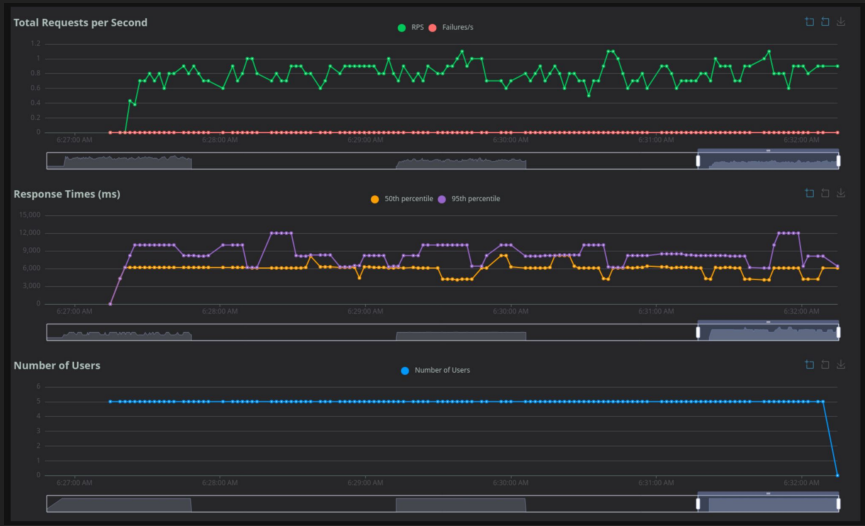


Loss on Purpose

At 3% send & receive loss:

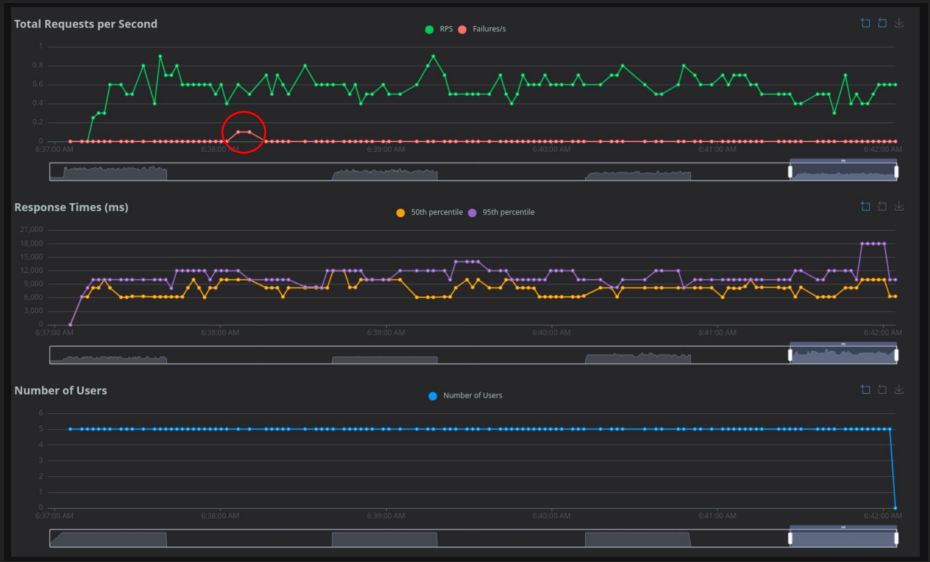


At 8% send & receive loss:

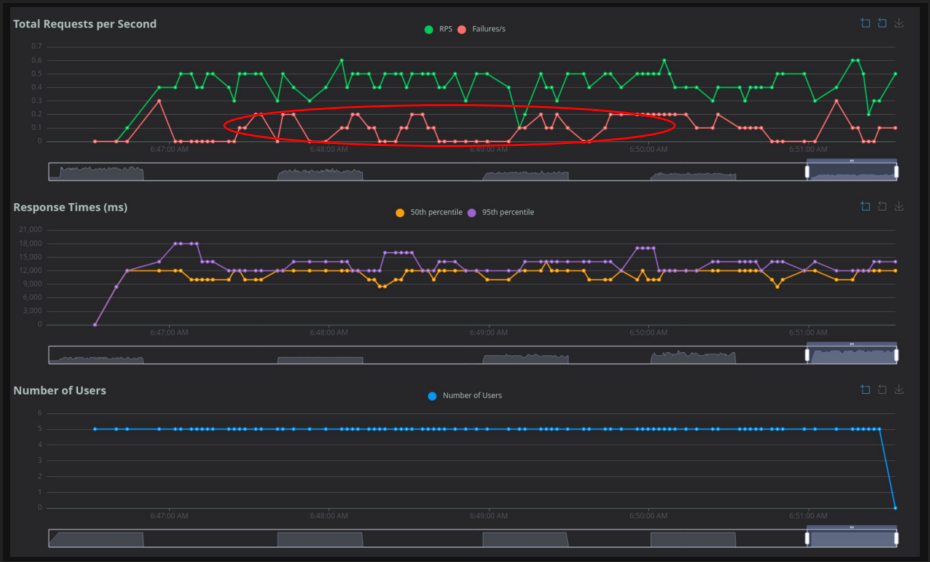


Loss on purpose

At 15% send & receive loss:



At 25% send & receive loss:



Loss on purpose

With loss, latency goes up significantly.

At around 15% loss, requests start to fail. This fail rate goes 17.8% at 25% loss.

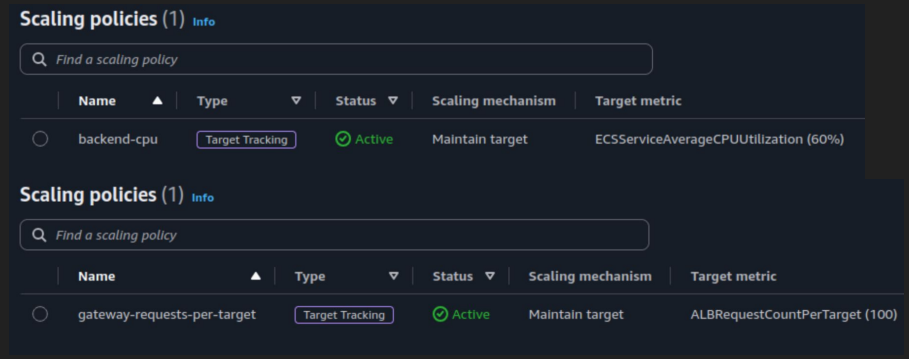
Loss	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
1%	Aggregated	469	0	3079.9	2034	6280	4096	1.57	0
3%	Aggregated	352	0	4135.63	2043	8298	4096	1.17	0
8%	Aggregated	249	0	5898.65	2044	12283	4096	0.83	0
15%	Aggregated	182	1	8011.59	4075	18225	4073.73	0.61	0
25%	Aggregated	129	28	11283.7	6135	18263	3216.28	0.43	0.09

Scaling Behavior

To see how it scales, we need to know the rule of scaling:

- Gateway starts to scale when requests per task goes to 100 per minute
- Backend workers only scales when CPU is above 60% utilizaiton

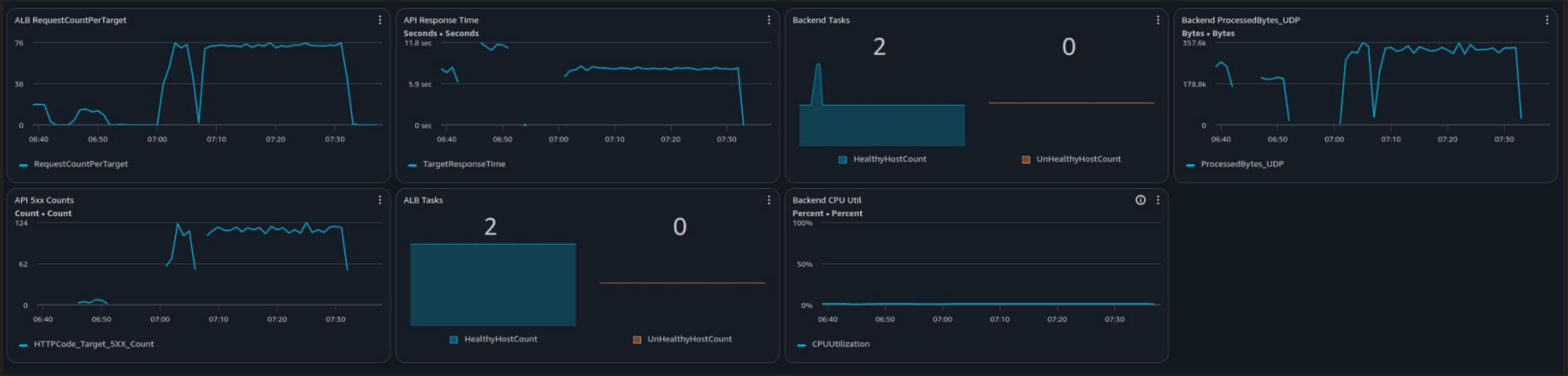
We will test with 20 users for 30 minutes on 4kb txt file.



Scaling Behavior

No scaling happened. The requests just failed and the 2 gateways can only handle 70 requests each. The backend cpu remained very low.

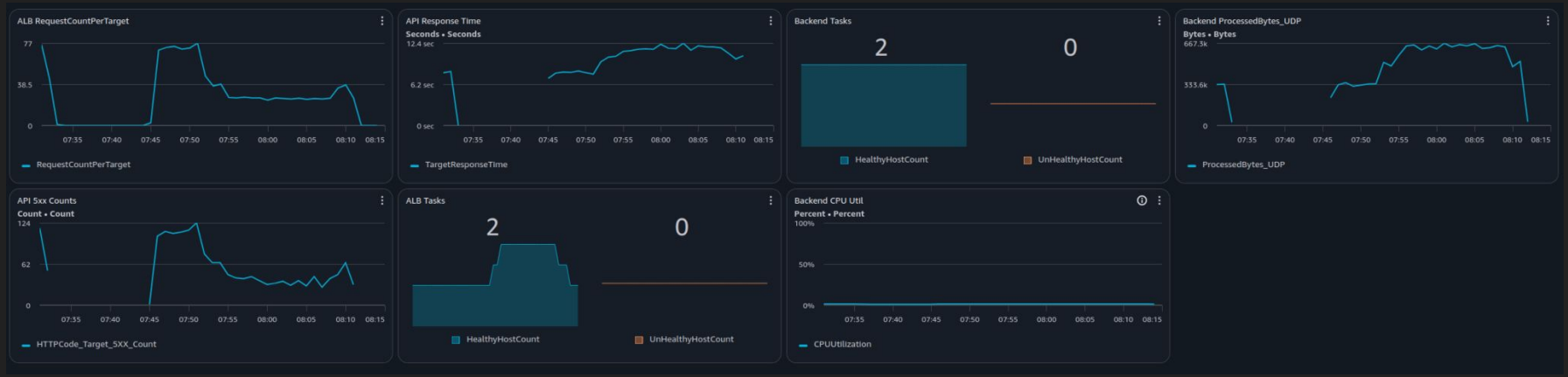
Changing the scaling rule will be necessary to observe changes.



Scaling Behavior

After changing the ALB scaling rule to 70 requests per task, scaling did happen with 20 workers.

Scaling in happened after about 15 minutes of low traffic. The failures dropped after introducing more tasks and the processed bytes also increased.



Scaling Behavior

Locust supported this by showing a better failure rate.

The real bottleneck is not the resources. With double gateway tasks, at least 10% requests still dies.



Questions?